TUGAS SOAL TEORI

Modul Bahan Ajar — Pemrograman Python

Struktur Data: List, Tuple, Set, Dictionary, dan Pengurutan

Nama	:	Eka Febryanto
NIM	:	20241320014
Kelas	:	A1 Sistem Informasi

Soal 1 — Perbedaan List dan Tuple

> *Jelaskan perbedaan mendasar antara list dan tuple dalam Python. Berikan contoh skenario di mana tuple lebih tepat digunakan daripada list, dan sebaliknya.*

Jawaban

Perbedaan mendasar antara list dan tuple terletak pada sifat **mutabilitas**. **List bersifat mutable** (dapat diubah setelah dibuat), sedangkan **tuple bersifat immutable** (tidak dapat diubah setelah dibuat). Keduanya sama-sama berurutan (*ordered*), mendukung pengindeksan dan *slicing*, serta dapat menyimpan elemen bertipe campuran.

Aspek	List	Tuple
Sintaks	`[1, 2, 3]`	`(1, 2, 3)` atau `1, 2, 3`
Mutabilitas	Mutable — elemen dapat ditambah, diubah, dan dihapus	Immutable — elemen tidak dapat diubah setelah dibuat
Metode bawaan	Banyak: `append()`, `remove()`, `pop()`, `sort()`, dll.	Hanya `count()` dan `index()`
Hashable	Tidak (tidak dapat menjadi kunci dictionary)	Ya, jika seluruh elemennya hashable
Memori & kecepatan	Relatif lebih besar dan lebih lambat dibuat	Lebih hemat memori dan umumnya lebih cepat dibuat
Tujuan umum	Kumpulan data yang berubah-ubah	Kumpulan data yang tetap dan terstruktur

Contoh perilaku keduanya:

```
daftar = [10, 20, 30]
daftar[0] = 99          # berhasil -> [99, 20, 30]
daftar.append(40)       # berhasil -> [99, 20, 30, 40]

tupel = (10, 20, 30)
tupel[0] = 99           # TypeError: 'tuple' object does not
                        # support item assignment
```

Skenario yang lebih tepat menggunakan tuple:

- **Data tetap yang tidak boleh berubah**, misalnya koordinat `(-6.9175, 107.6191)`, warna RGB `(255, 128, 0)`, atau nama hari dalam seminggu.
- **Kunci dictionary** yang terdiri dari beberapa nilai, misalnya `jarak[("Bandung", "Jakarta")] = 150`.
- **Mengembalikan beberapa nilai dari fungsi**, misalnya `return nilai_min, nilai_maks`.
- **Elemen dalam set**, karena set hanya menerima objek hashable.

Skenario yang lebih tepat menggunakan list:

- **Data yang sering berubah ukuran atau isinya**, misalnya keranjang belanja, daftar tugas (*to-do list*), atau antrean pelanggan.
- **Data yang perlu diurutkan atau dimodifikasi di tempat**, misalnya daftar nilai mahasiswa yang diurutkan menggunakan `sort()`.
- **Pengumpulan hasil dalam perulangan**, misalnya menambahkan hasil perhitungan satu per satu dengan `append()`.

Soal 2 — Set dan Pengindeksan

> *Mengapa set tidak mendukung pengindeksan seperti list dan tuple? Jelaskan kaitannya dengan implementasi hash table dan sifat unordered.*

Jawaban

List dan tuple disimpan sebagai **array berurutan** di memori: elemen ke-*i* menempati posisi ke-*i* yang pasti, sehingga `data[i]` bermakna dan dapat diakses langsung. Set tidak bekerja seperti itu.

Set diimplementasikan menggunakan **hash table**. Pada hash table, posisi penyimpanan suatu elemen **ditentukan oleh nilai hash elemen itu sendiri**, bukan oleh urutan pemasukannya. Secara sederhana, prosesnya adalah:

```
posisi = hash(elemen) % ukuran_tabel
```

Akibat dari mekanisme ini, terdapat beberapa konsekuensi yang menjelaskan mengapa pengindeksan tidak didukung:

- **Sifat unordered.** Tidak ada urutan logis antar elemen. Posisi elemen bergantung pada nilai hash, sehingga "elemen pertama" atau "elemen ke-2" tidak memiliki makna yang tetap.
- **Posisi tidak stabil.** Ketika elemen ditambahkan dan tabel perlu diperbesar (*resize*), seluruh elemen dihitung ulang posisinya. Dengan demikian, elemen yang tadinya berada di posisi tertentu dapat berpindah.
- **Tumbukan (collision).** Dua elemen berbeda dapat menghasilkan posisi yang sama dan ditangani oleh mekanisme resolusi tumbukan, sehingga letak akhir elemen makin tidak dapat diprediksi pemrogram.
- **Tujuan desain.** Set dirancang untuk **pengecekan keanggotaan yang cepat** dan **operasi himpunan** (union, intersection, difference), bukan untuk akses berdasarkan posisi.

```
buah = {"apel", "jeruk", "mangga"}
print(buah[0])      # TypeError: 'set' object is not subscriptable

# Solusi bila memerlukan urutan atau indeks:
daftar_buah = list(buah)      # ubah menjadi list
print(daftar_buah[0])         # sekarang dapat diindeks
print(sorted(buah))           # atau diurutkan -> list terurut
```

> **Catatan:** Sejak Python 3.7, `dict` menjamin urutan penyisipan, tetapi `set` **tidak** memberikan jaminan urutan apa pun. Karena itu, kode tidak boleh bergantung pada urutan iterasi set.

Soal 3 — Hashable

> *Apa yang dimaksud dengan hashable dalam konteks Python? Sebutkan tiga tipe data yang hashable dan tiga yang tidak hashable. Mengapa kunci dictionary harus hashable?*

Jawaban

Sebuah objek disebut **hashable** apabila memenuhi dua syarat berikut:

- Memiliki nilai hash (hasil `hash(obj)` melalui metode `__hash__()`) yang **tidak berubah selama masa hidup objek**.
- Dapat dibandingkan dengan objek lain (metode `__eq__()`), dan **dua objek yang sama nilainya harus memiliki nilai hash yang sama**.

Dalam praktiknya, objek immutable umumnya hashable, sedangkan objek mutable umumnya tidak hashable.

Tipe data hashable	Tipe data tidak hashable
`int`, misalnya `42`	`list`, misalnya `[1, 2, 3]`
`str`, misalnya `"python"`	`dict`, misalnya `{"a": 1}`
`tuple` (jika semua elemennya hashable), misalnya `(1, 2)`	`set`, misalnya `{1, 2, 3}`

Tipe hashable lain yang sering dijumpai antara lain `float`, `bool`, `None`, dan `frozenset`. Perlu diperhatikan bahwa tuple yang berisi objek mutable **tidak hashable**:

```
hash((1, 2))            # berhasil
hash((1, [2, 3]))       # TypeError: unhashable type: 'list'
hash([1, 2])            # TypeError: unhashable type: 'list'
```

Mengapa kunci dictionary harus hashable? Dictionary juga diimplementasikan dengan hash table. Saat menyimpan pasangan kunci–nilai, Python menghitung `hash(kunci)` untuk menentukan lokasi penyimpanan, lalu menggunakan hash yang sama untuk menemukan nilainya kembali. Agar mekanisme ini andal, nilai hash kunci harus **konstan**.

Jika kunci berupa objek mutable, isinya dapat berubah sehingga hash-nya ikut berubah. Python kemudian akan mencari di lokasi yang salah, dan pasangan kunci–nilai tersebut **tidak dapat ditemukan lagi** meskipun masih ada di dalam dictionary. Karena itu Python menolak kunci yang tidak hashable sejak awal:

```
d = {}
d[(1, 2)] = "valid"         # tuple -> berhasil
d["nama"] = "valid"         # str   -> berhasil
d[[1, 2]] = "tidak valid"   # TypeError: unhashable type: 'list'
```

Soal 4 — Kompleksitas Pencarian pada Set dan List

> *Jelaskan mengapa pencarian keanggotaan (operator in) pada set memiliki kompleksitas O(1) sedangkan pada list O(n). Apa implikasi praktisnya untuk data besar?*

Jawaban

Pada list, operator `in` melakukan **pencarian linear** (*linear search*): Python memeriksa elemen satu per satu dari awal hingga elemen ditemukan atau seluruh list habis diperiksa. Pada kasus terburuk, semua *n* elemen harus diperiksa, sehingga kompleksitasnya **O(n)**. Waktu pencarian bertambah sebanding dengan jumlah data.

Pada set, operator `in` memanfaatkan **hash table**. Python menghitung `hash(elemen)`, langsung menuju posisi yang sesuai, lalu memeriksa apakah elemen berada di sana. Jumlah langkah tidak bergantung pada banyaknya elemen, sehingga kompleksitas rata-ratanya **O(1)** (waktu konstan).

Struktur data	Kompleksitas `in`	Mekanisme
List	O(n)	Memeriksa elemen satu per satu secara berurutan
Set	O(1) rata-rata	Menghitung hash, lalu langsung menuju posisi elemen
Dictionary (kunci)	O(1) rata-rata	Sama dengan set, berbasis hash table

> **Catatan:** O(1) pada set adalah kompleksitas **rata-rata**. Pada kasus terburuk (banyak tumbukan hash), pencarian dapat menurun hingga O(n), tetapi hal ini sangat jarang terjadi pada penggunaan normal.

Ilustrasi perbandingan:

```
import time

data_list = list(range(1_000_000))
data_set = set(data_list)
target = 999_999            # elemen terakhir (kasus terburuk list)

mulai = time.perf_counter()
target in data_list         # O(n)
print("list:", time.perf_counter() - mulai)

mulai = time.perf_counter()
target in data_set          # O(1)
print("set :", time.perf_counter() - mulai)
```

Angka waktu yang tampil bergantung pada perangkat, tetapi pada data berukuran jutaan, pencarian pada set umumnya **jauh lebih cepat** (selisihnya bisa mencapai ribuan kali lipat).

Implikasi praktis untuk data besar:

- **Pencarian berulang sebaiknya memakai set.** Jika program perlu mengecek keanggotaan ribuan kali (misalnya memeriksa apakah ID pengguna sudah terdaftar), mengubah list menjadi set terlebih dahulu akan menghemat waktu secara signifikan.
- **Perulangan bersarang menjadi sangat mahal pada list.** Pencarian `in` di dalam *loop* berarti kompleksitas O(n × m) pada list, tetapi hanya sekitar O(n) bila menggunakan set.
- **Ada biaya awal.** Membuat set dari list membutuhkan waktu O(n) dan memori tambahan, sehingga konversi hanya sepadan jika pencarian dilakukan berkali-kali.
- **Ada konsekuensi desain.** Set tidak menyimpan duplikat, tidak mempertahankan urutan, dan hanya menerima elemen hashable. Jika urutan atau duplikat penting, list tetap diperlukan.

Soal 5 — sort() dan sorted()

> *Apa perbedaan antara metode sort() dan fungsi sorted() pada list? Kapan sebaiknya menggunakan masing-masing?*

Jawaban

Keduanya sama-sama mengurutkan data menggunakan algoritma yang stabil (*Timsort*) dan sama-sama mendukung parameter `key` serta `reverse`. Perbedaannya terletak pada cara kerja dan hasil yang dikembalikan.

Aspek	`list.sort()`	`sorted()`
Jenis	Metode milik list	Fungsi bawaan (*built-in*)
Cara kerja	**In-place**: mengubah list asli	Membuat **list baru**; data asli tidak berubah
Nilai kembalian	`None`	List baru yang sudah terurut
Objek yang didukung	Hanya list	Semua iterable (list, tuple, set, string, dictionary, dll.)
Penggunaan memori	Lebih hemat (tidak membuat salinan)	Membutuhkan memori tambahan untuk list baru

Contoh penggunaan:

```
angka = [5, 2, 9, 1]

# sorted(): data asli tetap, hasil berupa list baru
baru = sorted(angka)
print(baru)     # [1, 2, 5, 9]
print(angka)    # [5, 2, 9, 1]  -> tidak berubah

# sort(): data asli berubah, mengembalikan None
hasil = angka.sort()
print(angka)    # [1, 2, 5, 9]  -> berubah
print(hasil)    # None

# sorted() dapat dipakai pada iterable selain list
print(sorted((3, 1, 2)))          # tuple -> [1, 2, 3]
print(sorted({"b", "a", "c"}))    # set   -> ['a', 'b', 'c']
print(sorted([5, 2, 9], reverse=True))  # [9, 5, 2]
```

Kapan menggunakan sort():

- Ketika data asli **tidak lagi diperlukan** dalam urutan semula.
- Ketika data sangat besar dan **efisiensi memori** menjadi pertimbangan, karena tidak ada salinan yang dibuat.
- Ketika datanya sudah pasti berupa list.

Kapan menggunakan sorted():

- Ketika **urutan asli harus dipertahankan**, misalnya untuk dibandingkan atau digunakan kembali.
- Ketika data berupa **tuple, set, string, atau dictionary**, yang tidak memiliki metode `sort()`.
- Ketika hasil pengurutan langsung dipakai dalam ekspresi atau perulangan, misalnya `for x in sorted(data):`.

> **Kesalahan umum:** Menulis `data = data.sort()` akan membuat `data` bernilai `None` karena `sort()` tidak mengembalikan list. Gunakan `data.sort()` saja, atau `data = sorted(data)`.